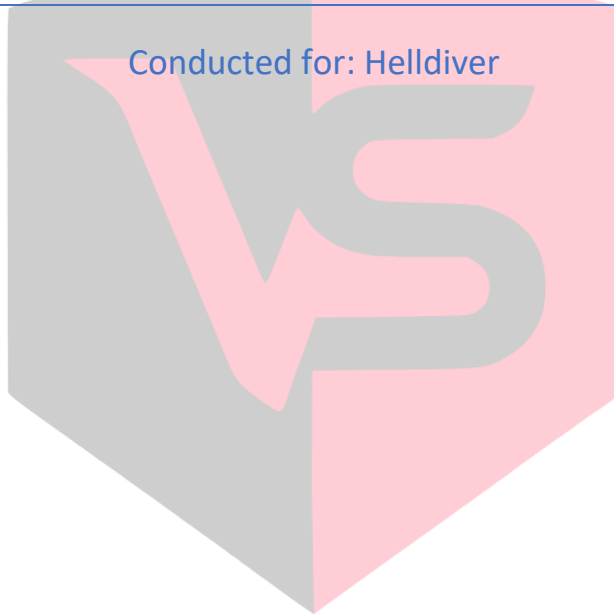




SMART CONTRACT AUDIT

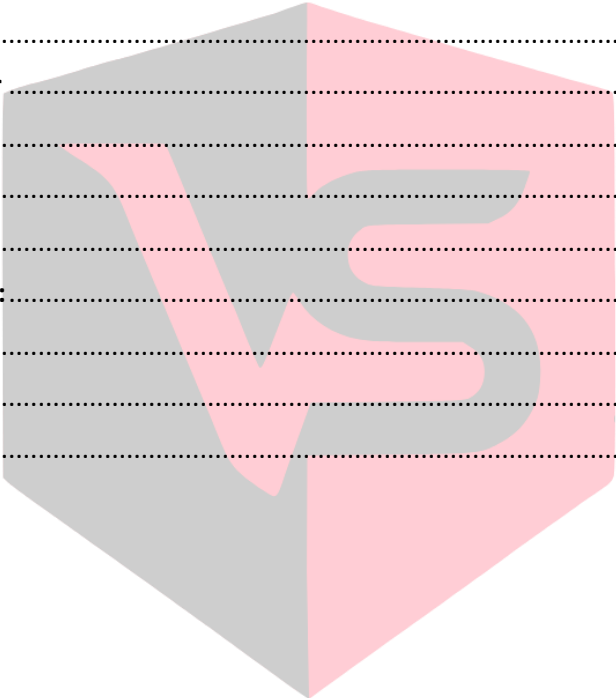
Conducted for: Helldiver



VULSIGHT

Contents

Executive Summary:	3
Project Background:	3
Audit Scope:	3
Audit Summary:	3
Severity Definitions:	5
Technical Checks:	5
Code Quality and Documentation:	5
Audit Findings:	6
Critical vulnerabilities:	6
Unstaking Logic Error	6
High vulnerabilities:	7
Pricing Mismatch	7
Minting Discrepancy:	8
Medium vulnerabilities:	10
Low vulnerabilities:	10
Disclaimer:	10
Conclusion:	11



VULSIGHT

Executive Summary:

The client contacted our Firm to conduct a smart contract audit on their Solidity based smart contracts. One of the contract was the basis for their ERC-20 Token, while the other one provides the functionalities to facilitate the presale of the tokens. In our comprehensive audit of the contracts, we carefully examined several critical aspects including functionality, security vulnerabilities, best practices, and smart contract efficiency. The contract meticulously initiates a token presale process, utilizing phased pricing to incentivize early participation while implementing features like automatic staking upon purchase. Despite its elaborate design to facilitate a structured presale through various stages, the audit identified a critical oversight in the Unstake function where tokens could potentially be unstaked unlimited times due to the omission of updating the `isunstake` flag. Moreover, the analysis revealed a mismatch between the stated pricing structure and the implemented pricing in the contract, which could lead to confusion and unintended outcomes. The Token Contract also had discrepancies with the intended tokens to be minted from the project specifications. The audit underlines the importance of rectifying these issues to uphold the integrity of the presale process and safeguard participant contributions against vulnerabilities.

Project Background:

The smart contracts are written in solidity. The token contract is a simple ERC20 Contract with no additional functionalities. The presale contract has owner functionality within it. The contracts are deployed on the Binance testnet.

Audit Scope:

Deployed Addresses	0x1364f0111b29bfcbbad854cd57c51cf560e8af7b 0xf73a03c035ceedb155abe8beed5cf964be3a200b
Chain	Binance TestNet
Language	Solidity
Audit Completion Date	15/4/24

Audit Summary:

- **Dynamic Pricing Mechanism:** The contract adopts a unique approach to pricing, utilising a predefined array of prices corresponding to distinct presale stages, influenced by the elapsed time since the presale launch. This strategy aims to reward early participants with lower prices. However, the actual prices defined in the contract do not align with typical price per token

formats and do not match the client-provided specifications, raising concerns over potential misinterpretation of pricing mechanics.

- **Multi-currency Contributions:** Contributions can be made in the native currency (e.g., BNB/ETH) and the ERC20 stablecoins USDT and USDC, providing flexibility to participants. The conversion to the equivalent token amounts leverages a real-time price feed for the native currency, yet for stablecoin contributions, it directly computes the number of tokens, showing an effective use of on-chain data and avoiding external dependency for stablecoin pricing.
- **Staking and Rewards:** The contract automatically stakes tokens purchased during the presale, with the APY and specific reward calculations encoded within. However, our audit identified a potential risk in the Unstake function—lacking a crucial update to the `isunstake` flag post-withdrawal, possibly allowing repeated unstaking. This oversight can lead to unintended exploitation, impacting the token's economic model and overall presale integrity.
- **Ownership and Administrative Functions:** The contract vests significant control in the owner for adjusting presale settings, token prices, and withdrawing funds. While such controls are common, they necessitate trust in the owner to act in the project's best interest. The presence of `onlyOwner` modifiers on sensitive functions mitigates unauthorized access risks.
- **Presale Status Management:** Dedicated functions to start and end the presale provide straightforward transitions between presale stages, including preparation for the open market.
- **Fund Withdrawal and Token Update Features:** Functionality allowing the owner to withdraw collected funds and update token addresses post-initial deployment introduces flexibility in managing and reallocating funds. While essential from an operational perspective, such features crucially require strong governance to prevent misuse.

We found:

- **1 Critical risk vulnerability**
- **2 High risk vulnerabilities**
- **0 Medium risk vulnerabilities**
- **0 Low risk vulnerabilities**

Severity Definitions:

Risk Level	Description
Critical	Any kind of vulnerability that could lead to direct token or monetary loss
High	Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.
Medium	Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss
Low	Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Matching Project Specifications	Not Passed
Logical Vulnerabilities	Not Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included two smart contracts, which are written in Solidity programming language. The code is well indented and clear in nature. The code had sufficient commenting but at some places within the code, the comments were incorrect.

Audit Findings:

Critical vulnerabilities:

One critical vulnerability was found in the smart contract.

Unstaking Logic Error

Summary

A critical vulnerability was identified in the Unstake functionality of the HELLDIVER presale contract, which could potentially be exploited to allow repeated unstaking of the same tokens, thus jeopardizing the integrity of the token economics and undermining the fairness of the reward distribution mechanism intended for staking participants.

Technical Overview

The contract incorporates an in-built staking mechanism whereby tokens purchased during the presale are immediately staked on behalf of the purchaser. Each stake entry is associated with a unique index for each user, containing information about staked tokens, rewards, and staking status, represented by the StakeData struct. The Unstake function is designed to allow users to withdraw their staked tokens along with any accrued rewards after the presale has ended.

A pivotal check within this function is intended to prevent the same stake from being unstaked more than once, achieved by assessing a boolean flag `isunstake` within the user's StakeData. Ideally, this flag should be set to true following a successful unstake operation to indicate that the stake has already been processed.

Observed Vulnerability

Upon inspection, it was noted that the Unstake function does not update the `isunstake` flag after tokens are unstaked. Specifically, after transferring the staked tokens and any rewards back to the user, the function neglects to set `userStake.isunstake` to true. Consequently, this omission enables the possibility of calling the Unstake function multiple times with the same stake index, each time transferring the same amount of tokens, which should not be permissible. Here is the vulnerable code:

```
function Unstake(uint256 _index) public {
    require(isPresaleEnded, "Presale not ended!");
    StakeData storage userStake = userStakes[msg.sender][_index];
    require(!userStake.isunstake, "Already withdrawn");
    uint256 _reward = calculateReward(msg.sender, _index);
    if (_reward > 0) {
        mainToken.transfer(msg.sender, _reward);
        userStake.claimedReward += _reward;
    }
    mainToken.transfer(msg.sender, userStake.stakedTokens);
    userStake.claimed_tokens = userStake.stakedTokens;
}
```

Potential Impact

- **Token Dilution:** Repeatedly unstaking the same tokens can lead to unwarranted dilution of the token supply, as users could exploit this oversight to claim more tokens than they are entitled to.
- **Compromised Economic Model:** The contract's economic model could be severely compromised, undermining the value and fairness of the presale and staking rewards.
- **Loss of Trust:** Exploitation of this vulnerability could result in a significant loss of trust among the community and stakeholders, potentially causing irreparable damage to the project's reputation and market performance.

Recommendation for Mitigation

To rectify this vulnerability, it is imperative that the contract code be amended to include the operation `userStake.isun stake = true`; within the `Unstake` function, immediately following the transference of staked tokens and rewards to the user. This action will appropriately mark the stake as processed and prevent any future attempts to unstake the same stake index.

High vulnerabilities:

Two high vulnerabilities were found in the smart contract.

Pricing Mismatch

Summary

A significant discrepancy was discovered in the presale contract regarding the implemented pricing structure versus the stated or expected pricing parameters for token presale stages in the specifications. This mismatch between the actual coded prices and the intended presale prices could lead to confusion among participants and potentially impact the fair valuation and distribution of tokens during the presale event.

Technical Overview

The contract employs a phased approach to pricing, with tokens offered at predetermined prices that vary according to the phase of the presale. These prices are stored in an array called `prices`, and the contract calculates the current presale phase based on the elapsed time since the `launchTime`. The purpose of this strategy is to incentivize early participation by offering lower prices in the initial stages, gradually increasing as the presale progresses.

Upon auditing the contract, it was observed that the prices encoded in the `prices` array significantly diverge from the prices specified or expected by the project stakeholders, as communicated in the audit request. The values in the `prices` array are markedly different from conventional token price formats and do not translate to the expected price-per-token values intended for each presale stage.

For instance, the expected pricing model presented for audit purposes outlines a clear linear progression in token pricing through the stages, such as 0.00001289 USD per token in Stage 1, scaling up sequentially to 0.00010000 USD per token in Stage 15. In contrast, the contract's prices array contains values that do not correspond to this progression or expected format even when being converted to BNB, leading to potential misalignments in token purchase values.

Observed Vulnerability

The affected code is present here:

```
function Unstake(uint256 _index) public {
    require(isPresaleEnded, "Presale not ended!");
    StakeData storage userStake = userStakes[msg.sender][_index];
    require(!userStake.isunstake, "Already withdrawn");
    uint256 _reward = calculateReward(msg.sender, _index);
    if (_reward > 0) {
        mainToken.transfer(msg.sender, _reward);
        userStake.claimedReward += _reward;
    }
    mainToken.transfer(msg.sender, userStake.stakedTokens);
    userStake.claimed_tokens = userStake.stakedTokens;
}
```

Potential Impact

- **Participant Confusion:** The mismatch in pricing could lead to confusion among participants, with actual purchase costs differing from expected prices.
- **Economic Discrepancy:** The valuation of tokens could be adversely affected, potentially impacting the economic model of the token and the presale's success.
- **Loss of Trust:** Discrepancies between stated and implemented pricing could harm the project's credibility and trust within the community, essential for a successful token launch.

Recommendation for Mitigation

It is recommended that the contract's prices array and the mechanisms that calculate and apply these prices to token sales during the presale be thoroughly reviewed and aligned with the initially stated pricing model. Clear documentation and transparency in communicating the pricing methodology to potential presale participants are also vital to mitigate confusion and maintain trust.

Minting Discrepancy:

Summary

A discrepancy has been identified within the HELLDIVER token's minting process as stipulated in the ERC20 token contract. The specified allocations for Exchange Liquidity Provision, CEX LISTING,

Marketing/Development, and Presale significantly deviate from the actual minted tokens as indicated in the contract's constructor. This discrepancy results in a substantial variance from the intended token distribution, which could severely impact the project's funding operations, market liquidity, and overall ecosystem development.

Technical Overview

The initial specifications for the distribution of minted tokens denote a strategic allocation aiming at supporting various operational, promotional, and liquidity assurance activities vital for the project's launch and sustainability:

- **Exchange Liquidity Provision:** 12.5% (6,250,000,000 tokens)
- **CEX LISTING:** 15% (7,500,000,000 tokens)
- **Marketing/Development:** 20% (included in Presale allocation)
- **Presale:** 52.5% (26,250,000,000 tokens)

However, the implementation in the HELLDIVER contract constructor reveals a token minting approach that conflicts with these specifications:

- **Presale Wallet:** 3,625,000,000 tokens
- **CEX Wallet:** 750,000,000 tokens
- **Liquidity Wallet:** 625,000,000 tokens

The minting function, `_mint`, is called with parameters that do not align with the stated distribution, signifying a considerable shortfall in the token amounts for each designated purpose.

Observed Vulnerability

The affected code is here:

```
contract HELLDIVER is ERC20 {
    constructor(
        address _presaleWallet,
        address _cexWallet,
        address _liquidityWallet
    ) ERC20("HELLDIVER Token", "HLD") {
        _mint(_presaleWallet, 3_625_000_000 * 10**decimals());
        _mint(_cexWallet, 750_000_000 * 10**decimals());
        _mint(_liquidityWallet, 625_000_000 * 10**decimals());
    }
}
```

Potential Impact

- **Insufficient Liquidity and Listing Funds:** The under-allocation of tokens for Exchange Liquidity Provision and CEX LISTING could jeopardize the project's ability to secure and maintain listings on exchanges, potentially limiting market access and liquidity.

- **Reduced Marketing and Development Budget:** The discrepancy affects the budget available for crucial marketing and development initiatives, possibly restricting the project's growth and adoption efforts.

Recommendation for Mitigation

Conduct a thorough review of the token contract to identify the root causes of the mismatch in minted token amounts. Amend the contract to reflect the initially outlined token distribution specifications accurately.

Medium vulnerabilities:

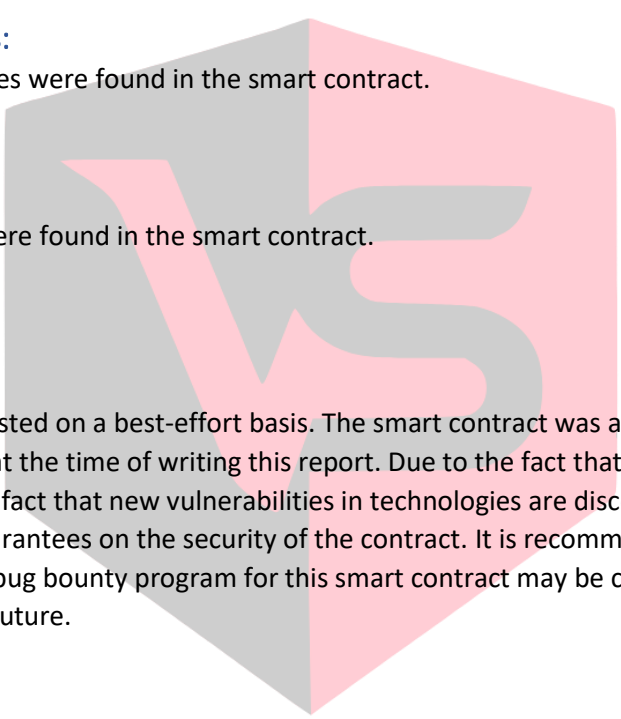
Zero medium vulnerabilities were found in the smart contract.

Low vulnerabilities:

Zero low vulnerabilities were found in the smart contract.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.



VULSIGHT

Conclusion:

The smart contract was tested extensively both automatically and manually. Few Significant Vulnerabilities were discovered within it. The vulnerabilities should be remediated before deployment to main net. It is recommended that additional security measures may be undertaken to ensure future security such that of a creation of a bug bounty program.

